

Atividade Complementar

Layouts no Flutter

Questão 1 - Column

A Column organiza widgets no eixo vertical, ou seja, um abaixo do outro. Esse componente é muito usado em telas de cadastro, perfis, telas de login, cartões de informação e menus. A lógica principal da Column é empilhar elementos verticalmente e controlar como eles se alinham e se distribuem nesse espaço.

Atributos importantes:

- children: lista de widgets que ficarão dentro da Column
- mainAxisAlignment: controla como os widgets serão distribuídos no eixo principal; na Column, o eixo principal é o vertical
- crossAxisAlignment: controla o alinhamento no eixo cruzado; na Column, o eixo cruzado é o horizontal
- mainAxisAlignment: define se a Column ocupará o menor espaço possível ou o máximo disponível
- verticalDirection: altera a direção da ordem vertical
- textDirection: pode influenciar alinhamentos em alguns cenários

Crie a tela inicial de um app de perfil profissional. Essa tela deve apresentar uma foto, nome, cargo e dois botões de ação organizados verticalmente.

```
+-----+
|               |
|           [ FOTO ]           |
|               |
|       Ana Oliveira           |
| Desenvolvedora Mobile       |
|               |
|   [ Seguir ]                 |
|   [ Mensagem ]               |
|               |
+-----+
```

- Crie uma Column centralizando os elementos na tela.
- Adicione um espaço vertical entre a foto, o nome, o cargo e os botões.
- Teste diferentes valores de mainAxisAlignment e observe o que muda quando você usa start, center, end, spaceAround, spaceBetween e spaceEvenly.

- d) Teste diferentes valores de `crossAxisAlignment` e observe a diferença entre `center`, `start`, `end` e `stretch`.
- e) Modifique a tela para que os textos fiquem alinhados à esquerda, mas os botões continuem centralizados.
- f) Faça uma segunda versão da tela ocupando toda a altura possível e outra versão ocupando apenas o espaço mínimo necessário. Compare os resultados usando `mainAxisSize`.
- g) Altere a ordem dos elementos visuais sem mudar a lista manualmente, explorando a ideia de direção vertical.

Questão 2 - Row

A `Row` organiza widgets no eixo horizontal, ou seja, lado a lado. É muito usada em barras de ações, cards, áreas com ícone e texto, menus inferiores, resumos financeiros e itens de corrida ou entrega.

Atributos importantes:

- `children`: lista de widgets dentro da `Row`
- `mainAxisAlignment`: controla a distribuição no eixo principal; na `Row`, o eixo principal é o horizontal
- `crossAxisAlignment`: controla o alinhamento vertical dos elementos
- `mainAxisSize`: define se a linha ocupa o máximo de largura ou apenas o necessário
- `textDirection`: altera a direção da leitura horizontal
- `verticalDirection`: interfere em certos alinhamentos

Crie a parte superior de uma tela de app de transporte, semelhante a um resumo de corrida. Você deve exibir um ícone do carro, o nome do motorista, a nota e um botão de contato.

```
+-----+
| [🚗] Carlos Silva ★ 4.9 [📞] |
+-----+
```

- a) Monte a estrutura principal usando `Row`.
- b) Ajuste o espaçamento dos elementos para que fiquem visualmente equilibrados.
- c) Teste `mainAxisAlignment` com `start`, `center`, `spaceBetween` e `spaceEvenly`.
- d) Aumente o tamanho do texto do nome e observe o comportamento da linha.
- e) Teste `crossAxisAlignment` com elementos de alturas diferentes, por exemplo um ícone pequeno e um botão maior.
- f) Crie uma segunda versão da mesma linha com os dados alinhados mais ao topo.
- g) Inverter a ordem dos itens para simular um layout alternativo e observar o efeito da direção horizontal.
- h) Transforme essa mesma linha em uma barra inferior com três ícones distribuídos igualmente.

Questão 3 - Container

O Container é um widget muito usado para dar forma visual a um elemento. Ele pode definir largura, altura, cor, margem, padding, alinhamento e decoração. É um dos widgets mais úteis para construir blocos visuais, cartões, banners, áreas clicáveis e seções de tela.

Atributos importantes:

- width: largura
- height: altura
- color: cor de fundo
- padding: espaçamento interno
- margin: espaçamento externo
- alignment: alinhamento do conteúdo interno
- decoration: estilização avançada, como borda, raio, sombra e gradiente
- child: widget interno

Crie um card de promoção para um app de delivery.

```
+-----+
|      30% OFF EM LANCHES      |
| Peça hoje e receba em 20 min |
|      [ PEDIR AGORA ]         |
+-----+
```

- Crie um Container com largura maior que a altura e com fundo colorido.
- Adicione padding para que o conteúdo não fique colado nas bordas.
- Adicione margin externa para afastar o card das laterais da tela.
- Centralize o conteúdo usando alignment.
- Estilize o Container com bordas arredondadas usando decoration.
- Adicione sombra ao card.
- Faça uma segunda versão com borda visível e sem cor sólida.
- Teste diferentes tamanhos de largura e altura para comparar proporção visual.
- Crie uma variação do card com o texto alinhado à esquerda e o botão na parte inferior.

Questão 4 - Padding e SizedBox

Padding adiciona espaço interno ao redor de um widget. SizedBox pode definir um tamanho fixo ou criar espaços vazios controlados entre elementos. Esses dois widgets são fundamentais para dar respiro visual ao layout.

Atributos importantes:

- padding: define o espaçamento interno, podendo usar all, symmetric, only
- Atributos importantes do SizedBox:
 - width: largura
 - height: altura
 - child: widget opcional interno

Crie uma tela simples de app de anotações com título, descrição e botão para salvar. A ideia é trabalhar espaçamento visual e organização.

```
+-----+
| Minhas anotações |
|                 |
| Título:         |
| [-----]      |
|                 |
| Descrição:      |
| [-----]      |
| [-----]      |
| [-----]      |
|                 |
|           [ SALVAR ]           |
+-----+
```

- Monte a estrutura da tela usando uma Column.
- Use Padding para afastar o conteúdo das bordas da tela.
- Use SizedBox para criar espaçamento vertical entre os campos.
- Teste EdgeInsets.all, EdgeInsets.symmetric e EdgeInsets.only.
- Faça uma versão com muito pouco espaçamento e outra com bastante espaçamento. Compare legibilidade e aparência.
- Use SizedBox(width: ...) em algum trecho horizontal da interface.
- Troque alguns SizedBox por Padding e compare o papel de cada um.
- Ajuste o layout para parecer mais com um formulário de aplicativo real.

Questão 5 - Center e Align

Center centraliza um widget no espaço disponível. Align também posiciona um widget, mas permite escolher a posição com mais precisão, como canto superior, canto inferior, centro à direita, centro à esquerda e assim por diante.

Atributos importantes:

- child: widget centralizado
- Atributos importantes do Align:
- alignment: define a posição do filho
- child: widget alinhado
- widthFactor: fator opcional de largura
- heightFactor: fator opcional de altura

Crie uma tela de app de streaming com uma capa central e um botão de favoritos no canto superior direito da área principal.

```
+-----+
|                                     |
|                                     | [♥] |
|                                     |
|      [ CAPA DO FILME ]            |
|                                     |
|      O Último Horizonte           |
|                                     |
+-----+
```

- Centralize a capa do filme usando Center.
- Posicione o botão de favorito no canto superior direito usando Align.
- Teste outros alinhamentos possíveis, como topo esquerdo, centro direito e canto inferior esquerdo.
- Crie uma segunda versão com o título centralizado e outra com o título alinhado à esquerda.
- Teste o uso de Align em áreas com tamanhos diferentes para observar a mudança na posição relativa.
- Crie uma área de destaque em que a imagem fique centralizada, mas um selo "Novo" fique no canto inferior direito.
- Compare o comportamento de Center e Align explicando no código, com comentários, quando cada um faz mais sentido.

Questão 6 - Expanded e Flexible

Expanded e Flexible são usados dentro de Row e Column para controlar como os widgets ocupam o espaço disponível. Expanded tende a forçar o preenchimento do espaço livre; Flexible permite maior adaptação ao conteúdo.

Atributos importantes:

- child: widget interno
- flex: quantidade proporcional de espaço
- fit: no Flexible, define como o widget ocupará o espaço disponível

Crie uma tela de app financeiro com três cartões de resumo lado a lado: saldo, receitas e despesas.

```
+-----+
| [ Saldo ] [ Receitas ] [ Despesas ] |
+-----+
```

Depois, crie uma segunda versão em coluna:

```
+-----+
| [ Saldo ] |
| [ Receitas ] |
| [ Despesas ] |
+-----+
```

- Monte os três cartões dentro de uma Row.
- Use Expanded para fazer os três ocuparem a largura disponível igualmente.
- Teste flex com proporções diferentes, por exemplo 2, 1 e 1.
- Faça uma versão em que o card de saldo fique maior que os demais.
- Substitua alguns Expanded por Flexible e observe a diferença.
- Coloque textos maiores em um dos cards e veja como o layout reage.
- Crie uma versão com espaçamento entre os cards.
- Leve a mesma lógica para uma Column e veja como o preenchimento do espaço muda no eixo vertical.
- Escreva no código qual situação ficou mais apropriada para Expanded e qual ficou melhor com Flexible.

Questão 7 - Stack e Positioned

Stack permite sobrepor widgets em camadas. Positioned é usado dentro da Stack para posicionar elementos com coordenadas relativas às bordas. Essa combinação é muito útil para banners, avatares com selo, cards com ícones flutuantes, notificações e sobreposição de texto em imagem.

Atributos importantes:

- children: lista de widgets empilhados
- alignment: alinhamento padrão das camadas
- clipBehavior: comportamento ao ultrapassar limites
- Atributos importantes do Positioned:
 - top: distância do topo
 - bottom: distância da base
 - left: distância da esquerda
 - right: distância da direita
 - child: widget posicionado

Crie um card de produto para um app de loja virtual. O card deve ter imagem principal, nome do produto, preço e um selo de desconto sobreposto no canto superior direito.

```
+-----+
|               [ -20% ]               |
|                                     |
|      [ IMAGEM PRODUTO ]              |
|                                     |
|      Tênis Running Pro                 |
|      R$ 299,90                         |
+-----+
```

- Use Stack para sobrepor o selo de desconto na imagem.
- Posicione o selo usando Positioned.
- Teste o selo em outras posições, como canto superior esquerdo e canto inferior direito.
- Adicione um ícone de favorito em outra camada.
- Faça uma versão com texto por cima da imagem, como um banner promocional.
- Teste diferentes tamanhos da imagem para ver como os posicionamentos reagem.
- Crie uma segunda tela no estilo app de música, com capa do álbum e botão de play sobreposto ao centro.
- Experimente retirar o Positioned de um elemento e observe a diferença de comportamento.

Questão 8 - Wrap

Wrap organiza widgets em sequência horizontal, mas quando falta espaço ele automaticamente quebra para a próxima linha. É muito útil para categorias, filtros, tags, chips e botões pequenos.

Atributos importantes:

- children: lista de widgets
- spacing: espaço horizontal entre itens
- runSpacing: espaço vertical entre linhas
- alignment: alinhamento dos itens em cada linha
- runAlignment: alinhamento entre linhas
- direction: direção principal do fluxo

Crie uma tela de app de busca de restaurantes com filtros de comida.

```
+-----+
| [Pizza] [Japonesa] [Vegana]      |
| [Hambúrguer] [Árabe] [Café]     |
| [Sobremesa] [Massas]            |
+-----+
```

- Monte a lista de filtros usando Wrap.
- Defina espaçamento horizontal com spacing.
- Defina espaçamento vertical com runSpacing.
- Teste a tela com poucos filtros e depois com muitos filtros.
- Teste alignment para observar como os itens se distribuem em cada linha.
- Teste larguras de tela diferentes, simulando telas menores.
- Estilize cada filtro como um botão ou Chip.
- Crie uma segunda versão para um app de cursos com categorias como Flutter, Dart, UI, Backend e Banco de Dados.
- Compare, no comentário do código, por que Wrap funciona melhor que Row nesse caso.

Questão 9 - ListView

A ListView é um widget usado para exibir uma lista rolável de elementos. Ela é ideal quando o conteúdo pode ultrapassar a altura da tela. Em apps reais, aparece em listas de mensagens, produtos, notificações, tarefas, corridas, músicas e muitos outros contextos.

Atributos importantes:

- children: lista de widgets da ListView
- scrollDirection: direção da rolagem
- padding: espaçamento interno da lista
- itemExtent: altura fixa para os itens, em alguns casos
- shrinkWrap: ajusta o tamanho ao conteúdo em contextos específicos
- physics: comportamento da rolagem

Crie uma tela de app de tarefas com uma lista de atividades do dia.

```
+-----+
| Minhas tarefas |
|-----|
| [ ] Estudar Flutter |
| [ ] Fazer exercícios |
| [ ] Revisar layout |
| [ ] Entregar atividade |
| [ ] Ler documentação |
| [ ] Atualizar projeto |
| |
| (scroll) |
|-----+
```

- Crie uma ListView contendo pelo menos 10 tarefas.
- Adicione espaçamento interno usando padding.
- Teste a rolagem vertical padrão.
- Crie uma segunda versão com rolagem horizontal.
- Faça cada item da lista parecer um card simples.
- Teste a lista com itens muito curtos e itens com textos longos.
- Adicione ícones no início de cada item.
- Observe o que acontece quando você tenta usar muitos itens dentro de uma Column sem ListView, e compare com o uso correto da lista.
- Faça uma versão visualmente parecida com um app de compras, trocando tarefas por produtos.

Questão 10 - SingleChildScrollView

O `SingleChildScrollView` permite rolagem para um único widget filho. Normalmente ele é usado quando você tem uma estrutura única, como uma `Column`, e quer permitir que todo o conteúdo role quando ultrapassar a tela.

Atributos importantes:

- `child`: widget único que receberá rolagem
- `scrollDirection`: direção da rolagem
- `padding`: pode ser combinado com widgets internos
- `physics`: comportamento da rolagem

Crie uma tela de app de cadastro com muitos campos. A tela deve poder rolar completamente sem quebrar o layout.

```
+-----+
| Cadastro |
+-----+
| Nome |
| [-----] |
| Email |
| [-----] |
| Telefone |
| [-----] |
| Endereço |
| [-----] |
| Cidade |
| [-----] |
| Estado |
| [-----] |
| Senha |
| [-----] |
| Confirmar senha |
| [-----] |
| |
| [ CRIAR CONTA ] |
| |
| (scroll) |
+-----+
```

- a) Monte a tela usando uma Column como estrutura principal.
- b) Envolver essa Column com SingleChildScrollView.
- c) Adicione espaçamento entre os campos com SizedBox.
- d) Adicione Padding nas bordas da tela.
- e) Teste o comportamento em uma tela menor.
- f) Faça uma versão sem rolagem e observe o problema visual.
- g) Crie uma segunda tela no estilo app de delivery, com muitas informações de endereço e pagamento.
- h) Teste a rolagem horizontal para entender que o widget também suporta outra direção.
- i) Escreva no código por que SingleChildScrollView faz sentido aqui, enquanto ListView faz mais sentido quando há repetição de itens.

Questão 11 - ListTile

O ListTile é um widget pronto para representar itens de lista de forma organizada. Ele costuma trazer área para ícone, título, subtítulo e ações laterais. É muito usado em menus, configurações, listas de contatos, chats e notificações.

Atributos importantes:

- leading: widget à esquerda
- title: título principal
- subtitle: texto secundário
- trailing: widget à direita
- onTap: ação ao tocar
- tileColor: cor do item
- contentPadding: espaçamento interno
- dense: reduz altura visual
- isThreeLine: permite mais linhas de conteúdo

Crie uma tela de configurações de um app.

```
+-----+
| Configurações                               |
|-----|
| [🔔] Notificações                        [ > ] |
|      Ativar alertas do app                |
|-----|
| [🌙] Tema escuro                        [ > ] |
|      Ajustar aparência                    |
|-----|
| [🔒] Privacidade                        [ > ] |
|      Gerenciar permissões                  |
|-----+
```

- Crie uma lista com pelo menos 5 opções usando ListTile.
- Use leading, title, subtitle e trailing em todos os itens.
- Teste um item com apenas título e outro com título mais subtítulo.
- Altere a cor de fundo de alguns itens.
- Teste diferentes espaçamentos com contentPadding.
- Crie uma versão mais compacta usando dense.
- Faça uma segunda tela de app de mensagens usando ListTile para representar conversas.
- Adicione um avatar no leading e um horário no trailing.
- Faça comentários no código sobre por que ListTile acelera a montagem de listas comuns.

Questão 12 - Row + Column combinados

Row e Column raramente aparecem sozinhos em apps reais. Na maioria das vezes, você combina os dois para criar áreas complexas, como cards, perfis, listagens, menus e painéis.

Atributos importantes:

- todos os atributos de Row
- todos os atributos de Column
- relação entre eixo principal e eixo cruzado em cada widget
- uso conjunto com Expanded, Padding, Container e SizedBox

Crie um card de motorista de app de transporte com foto, nome, avaliação, veículo e botão de chamada.

```
+-----+
| [FOTO] Mariana Souza |
|      ★ 4.8           |
|      Honda Civic - Preto |
|                        | [ Ligar ] |
+-----+
```

- Use uma Row como estrutura principal do card.
- Dentro dessa Row, use uma Column para organizar nome, nota e veículo.
- Adicione um botão na lateral direita.
- Use SizedBox para espaçar a foto do bloco de textos.
- Use crossAxisAlignment na Column para alinhar os textos à esquerda.
- Teste a troca de alinhamentos na Row para observar o impacto visual.
- Faça uma segunda versão desse card para um app de restaurante, trocando foto do motorista por foto do prato.
- Crie uma versão com texto longo para o nome do veículo e veja se precisa usar Expanded ou Flexible.
- Explique no código por que combinar Row e Column é mais poderoso do que usar apenas um layout isolado.

Questão 13 - AppBar + Body estruturado

Embora a AppBar não seja um widget de layout puro como Row e Column, ela faz parte da estrutura visual de muitas telas e precisa ser pensada junto com o restante do layout. Trabalhar AppBar com um body bem organizado ajuda a entender a composição completa de uma tela.

Atributos importantes da AppBar:

- title: título principal
- leading: widget à esquerda
- actions: ações à direita
- centerTitle: centralização do título
- backgroundColor: cor de fundo
- elevation: profundidade visual

Crie a tela inicial de um app de notícias.

```
+-----+
| [≡] Notícias Hoje           [🔍] |
|-----|
| Destaque do dia             |
| [ imagem/banner ]          |
|                             |
| Últimas notícias           |
| - Título 1                  |
| - Título 2                  |
| - Título 3                  |
|                             |
+-----+
```

- Crie um Scaffold com AppBar.
- Adicione um ícone no leading.
- Adicione pelo menos uma ação em actions.
- Monte o body usando Column e outros widgets de layout.
- Crie uma área de destaque com Container.
- Adicione uma lista de manchetes abaixo do destaque.
- Faça uma segunda versão com o título centralizado.
- Altere cor, elevação e ícones para criar outro estilo de app, como música ou delivery.
- Organize a tela para ficar visualmente equilibrada entre topo e conteúdo principal.

Questão 14 - Tela com cards em sequência

Uma tela com cards em sequência é um exercício excelente para treinar repetição visual, espaçamento e composição entre Container, Column, Row, Padding e SizedBox.

Crie uma tela de app bancário com cards de opções rápidas.

```
+-----+
| Olá, João                   |
|-----|
| [ Saldo ]                   |
| [ Transferir ]              |
| [ Pagar boleto ]            |
| [ Cartões ]                  |
|                             |
+-----+
```

- a) Crie uma tela com um cumprimento no topo.
- b) Logo abaixo, monte pelo menos 4 cards empilhados verticalmente.
- c) Cada card deve ter um ícone, um título e uma descrição curta.
- d) Use Container para estilizar os cards.
- e) Use Row e Column dentro de cada card.
- f) Adicione espaçamentos consistentes entre os cards.
- g) Faça uma segunda versão horizontal com cards lado a lado.
- h) Compare qual organização funciona melhor para telas menores.
- i) Crie uma versão alternativa com contexto de app acadêmico, usando opções como notas, faltas, horários e atividades.

Questão 15 - Layout de perfil completo

Esse exercício integra vários widgets para montar uma tela mais completa. O objetivo é trabalhar composição real, hierarquia visual e distribuição de áreas na interface.

Crie uma tela de perfil de rede social.

```
+-----+
|          [ FOTO ]          |
|          Juliana Costa      |
|          UI/UX Designer     |
|                              |
| [ Seguindo ] [ Seguidores ] |
|    250          1800        |
|                              |
|          [ Editar perfil ]  |
|                              |
| Sobre mim                   |
| Apaixonada por design e mobile |
+-----+
```

- a) Centralize foto, nome e cargo no topo.
- b) Crie uma Row para exibir estatísticas de seguidores.
- c) Faça cada estatística usar uma Column interna.
- d) Adicione um botão centralizado abaixo das estatísticas.
- e) Crie a seção “Sobre mim” alinhada à esquerda.
- f) Use Padding e SizedBox para melhorar o espaçamento.
- g) Faça uma segunda versão com o botão alinhado à largura total usando stretch ou outra estratégia equivalente.
- h) Adicione uma pequena faixa no topo com cor diferente, simulando capa de perfil.
- i) Transforme essa mesma tela em perfil de entregador, médico, professor ou artista.